

T4 Spring Boot May 2026 Set - 2

Question 1

Your high-traffic e-commerce microservice needs to call a legacy inventory system to check stock before completing an order. Normally, the legacy system responds in 50ms. However, during a flash sale, the legacy system becomes overwhelmed and starts taking 15 seconds to respond. Because your Tomcat threads are stuck waiting for the legacy system, your service quickly runs out of worker threads, causing your service to crash and take down the entire checkout flow. A junior developer added a Spring `@Retryable` annotation to the method, but the infrastructure team reports this actually made the outage much worse.

What is the correct architectural pattern to implement here to protect your microservice and prevent cascading failures?

- **A)** Change the external HTTP call to use `CompletableFuture.runAsync()`. This offloads the wait time to a background daemon thread pool, guaranteeing the main Tomcat threads are never blocked.
- **B)** Implement a Circuit Breaker pattern (using a library like `Resilience4j`). Configure it to "open" the circuit if the slow-call rate exceeds a certain threshold, which will immediately fail-fast (or return a fallback response) without hitting the legacy system, giving the legacy system time to recover.
- **C)** Replace the synchronous REST call with a Message Queue (like RabbitMQ). Publish the stock check request to an exchange and use a `@RabbitListener` to wait for the response, decoupling the systems completely.
- **D)** Increase the `server.tomcat.threads.max` property in your `application.yml` from 200 to 5000 to absorb the delayed responses and keep the checkout flow active.

Answer: B

Explanation: This is the textbook use case for the Circuit Breaker pattern. When a downstream system degrades, retrying (what the junior dev did) acts like a DDoS attack, making the overwhelmed system fail faster. A Circuit Breaker detects the high latency or error rate and trips the circuit open. Subsequent calls fail immediately (or return a cached/fallback value) without waiting 15 seconds, instantly freeing up your Tomcat threads and preventing the cascading failure.

Question 2

You are optimizing a Spring Data JPA application. You have an API endpoint `GET /api/users/summaries` that needs to return a list of 50,000 users for a reporting dashboard. The frontend only needs three fields: `id`, `firstName`, and `email`.

Currently, the service calls `userRepository.findAll()`. The `User` entity has 35 mapped columns, including a large `@Lob` profile picture and several `@OneToMany(fetch = FetchType.EAGER)` relationships. Calling this endpoint regularly crashes the JVM with a `java.lang.OutOfMemoryError`.

What is the most performant and idiomatic Spring Data JPA solution to resolve this issue without altering the global `User` entity configuration?

- **A)** Use Java Streams to map the heavy entities to a lightweight DTO:
`userRepository.findAll().stream().map(UserSummaryDTO::new).collect(Collectors.toList())`

- **B)** Annotate the service method with `@Transactional(readOnly = true)`. This prevents Hibernate from hydrating the objects into the First-Level Cache, bypassing the memory overhead entirely.
- **C)** Create a Spring Data Projection (either an interface like `UserSummary` or a record/DTO) and define a repository method: `List<UserSummary> findBy()`; Spring will automatically generate an optimized SQL query that `SELECTs` only those three specific columns.
- **D)** Use `@Query` to write a native SQL query, return a `List<Object[]>`, and manually iterate through the array indices in a for loop to construct your JSON response in the controller.

Answer: C

Explanation: Spring Data Projections are designed exactly for this scenario. If you fetch the whole entity (Option A), Hibernate still pulls all 35 columns and eager associations across the network and into memory before your Java Stream can filter them, which won't fix the OOM error. By using an interface or DTO projection, Spring Data alters the underlying Hibernate SQL to `SELECT id, first_name, email FROM users`, drastically reducing database I/O, network transit, and JVM heap usage.

Question 3

Your Spring Boot backend processes millions of financial transactions daily via a file parsing service. Each row contains a `transactionType` string. Business logic dictates there are only 4 possible transaction types globally (e.g., "DEPOSIT", "WITHDRAWAL", "FEE", "TRANSFER").

When analyzing a JVM heap dump during a performance audit, you discover that the application is holding 2.5 million distinct String objects in memory. Upon closer inspection, 600,000 of these objects contain the exact text "DEPOSIT", consuming gigabytes of RAM unnecessarily.

Which Core Java feature should you use during the parsing phase to force the JVM to deduplicate these strings and drastically reduce heap usage?

- **A)** Instantiate the strings using the `StringBuilder` class instead of the `String` class, as `StringBuilder` bypasses the immutable string constraints and reuses memory blocks.
- **B)** Call the `.intern()` method on the parsed string before assigning it to your DTO. This forces the JVM to check the String Constant Pool; if "DEPOSIT" already exists, it discards the new object and returns the memory reference to the pooled instance.
- **C)** Declare the `transactionType` field in your DTO with the `transient` keyword so the Garbage Collector aggressively removes the string from memory as soon as the parsing loop finishes.
- **D)** Increase the JVM Metaspace size using the `-XX:MaxMetaspaceSize` flag to give the Java compiler more room to cache string literals.

Answer: B

Explanation: This tests deep knowledge of JVM memory management. Because the strings are parsed dynamically at runtime (from a file/network), the JVM creates a brand new `String` object in the heap for every single row, even if the text is identical. Calling `String.intern()` tells the JVM: "If a string with this exact character sequence already exists in the String Pool, give me that reference instead." This collapses 600,000 separate "DEPOSIT" objects down to exactly 1 object in memory.

Question 4

You have a singleton `@Service` bean named `ReportGeneratorService`. It depends on a `@Component` named `CorrelationIdGenerator`. Because every report needs a strictly unique ID, you annotated `CorrelationIdGenerator` with `@Scope("prototype")`, ensuring a new instance is created every time it is requested.

In your service, you use constructor injection to autowire the prototype bean. You have a method `generateReport()` that calls `correlationIdGenerator.getNewId()`.

When 100 users call the API concurrently, they all receive a report with the exact same Correlation ID.

Why did the prototype scope fail to provide a unique instance for each request, and how do you fix it?

- **A)** Prototype beans are not thread-safe. Concurrent HTTP threads are causing a race condition on the internal ID generation algorithm. The fix is to use `AtomicLong` inside the prototype bean.
- **B)** The `@Scope` annotation is ignored by Spring unless the bean is explicitly requested via `ApplicationContext.getBean()`. Constructor injection always defaults to Singleton.
- **C)** This is the "Prototype injected into Singleton" problem. The Singleton service is instantiated exactly once during application startup. Its dependencies are resolved exactly once. The prototype bean is created once, injected into the singleton, and cached there forever. The fix is to use Method Injection (`@Lookup`) or inject an `ObjectFactory<CorrelationIdGenerator>`.
- **D)** Spring Boot 3.x removed support for the Prototype scope to optimize startup times. You must implement the `FactoryBean` interface to manually control instantiation logic.

Answer: C

Explanation: This is a very common architectural trap. Defining a bean as a prototype means Spring will create a new instance *every time it is asked for one*. However, a Singleton bean only "asks" for its dependencies once—when the Singleton itself is created at startup. Therefore, the Singleton holds a single, permanent reference to one specific prototype instance. To get a new instance dynamically at runtime within a singleton, you must use `@Lookup` or inject an `ObjectFactory<T>/Provider<T>` and call `.getObject()` when needed.

Question 5

You are designing a REST API for a payment gateway. The endpoint is `POST /api/v1/charges`.

A mobile application sends a request to charge a user's credit card \$50. Your server processes the payment successfully with Stripe and saves the record to the database. However, as your server sends the HTTP 200 OK response, the user's mobile network drops.

Because the mobile app never received the success response, its internal retry logic kicks in and automatically fires the exact same POST request again 3 seconds later. Your server processes it again, double-charging the customer.

What is the industry-standard REST API pattern you must implement to prevent this double-charging scenario during network retries?

- **A)** Change the HTTP method from POST to PUT. PUT requests are intercepted by the TCP networking layer and automatically deduplicated before reaching the Tomcat server.

- **B)** Require the client to generate and send a unique Idempotency-Key in the HTTP headers (e.g., a UUID). The server stores this key. If a request arrives with an Idempotency Key that has already been processed successfully, the server skips the payment logic and simply returns the cached success response.
- **C)** Implement a distributed database lock using Redis. Lock the user's account ID for 30 seconds during any payment processing to strictly reject concurrent or immediate subsequent requests.
- **D)** Return an HTTP 409 Conflict status code immediately if the same IP address makes two identical POST requests within a 5-second window.

Answer: B

Explanation: This scenario deals with API Idempotency. POST is inherently non-idempotent (calling it twice creates two resources). Network failures guarantee that clients *will* retry requests. To safely handle retries on non-idempotent operations like payments, the standard practice (used by companies like Stripe and PayPal) is the Idempotency Key. The client generates the key for the transaction. If the network drops and the client retries, it sends the *same* key. The server sees the key, realizes it already did the work, and safely returns the previous result without double-charging.

Question 6

Your e-commerce application has transitioned to a microservices architecture. When a user clicks "Checkout", the API Gateway routes the request to the OrderService. The OrderService synchronously calls the PaymentService via REST to charge the credit card, which succeeds. Next, the OrderService calls the InventoryService to reserve the items, but the InventoryService returns a 400 Bad Request because another user bought the last item a millisecond prior.

Because the database tables for Orders, Payments, and Inventory are in completely separate physical databases, you cannot use a simple @Transactional annotation to roll back the payment. What is the standard architectural pattern to resolve this distributed transaction failure?

- **A)** Implement the Two-Phase Commit (2PC) protocol using an XA-compliant Transaction Manager. The Transaction Manager will hold a lock on the Payment database row until the Inventory database confirms it is ready to commit, ensuring strict ACID compliance across the network.
- **B)** Implement the Saga Pattern. The OrderService must publish an InventoryFailedEvent. A compensation listener inside the PaymentService will catch this event and automatically execute a reverse transaction (a refund) to undo the original charge, bringing the system back to eventual consistency.
- **C)** Wrap the external REST calls in a try-catch block inside the OrderService. In the catch block, use Spring Data's PlatformTransactionManager to manually invoke rollback() on the injected DataSource beans for the other microservices.
- **D)** Change the HTTP calls from synchronous REST to asynchronous WebSockets. WebSockets maintain a persistent, bidirectional TCP connection, which allows Spring Boot to cascade the HTTP connection drops as database rollback signals across the cluster.

Answer: B

Explanation: Microservices cannot share database transactions. Option A (2PC/XA) is an older pattern that is heavily discouraged in modern cloud-native architectures because it blocks resources and scales poorly across slow networks. Option B, the Saga Pattern, is

the industry standard. Instead of holding locks across services, each service executes a local transaction. If a downstream service fails, it triggers a series of "compensating transactions" (e.g., refunds, cancellations) in the upstream services to undo the work.

Question 7

You are building an in-memory caching mechanism for a heavily used Core Java application. You define a standard `HashMap<UserSession, UserProfile>` cache = new `HashMap<>()`.

When a user logs in, their `UserSession` object is created and added as a key, with their profile as the value. When the user clicks "Log Out", the application destroys all active references to the `UserSession` object in the controllers and security context, expecting the JVM to free up the memory.

However, after three days in production, the application crashes with a `java.lang.OutOfMemoryError`.

Why did the Garbage Collector fail to remove the logged-out users' objects from memory, and what specific Java Collection must you use to fix this?

- **A)** The Garbage Collector only runs when the CPU is idle. Because the application was under heavy load for three days, the GC never had a chance to execute the "Mark and Sweep" algorithm. The fix is to programmatically call `System.gc()` when the user logs out.
- **B)** Even though the application destroyed its active references to the `UserSession` object, the `HashMap` itself still holds a strong memory reference to both the key and the value. As long as the `HashMap` exists, the GC cannot collect the objects inside it. The fix is to replace the `HashMap` with a `WeakHashMap`.
- **C)** The `UserProfile` object likely contains circular references (e.g., `Profile` points to `Address`, and `Address` points back to `Profile`). The JVM's Garbage Collector cannot resolve circular dependencies and leaves them in memory indefinitely. The fix is to use a `ConcurrentHashMap` to sever the links.
- **D)** `HashMap` instances automatically serialize their contents to the OS page file when they grow too large. The memory error is actually a Disk I/O error masked by the JVM. The fix is to wrap the map in `Collections.synchronizedMap()`.

Answer: B

Explanation: This describes a classic Java memory leak. A standard `HashMap` holds "strong references" to its keys and values. Even if you nullify the `UserSession` everywhere else in your app, the map still points to it, so the Garbage Collector cannot touch it. A `WeakHashMap`, however, holds "weak references" to its keys. If the *only* thing pointing to a `UserSession` is the `WeakHashMap`, the GC is allowed to destroy the object and automatically remove the entry from the map, making it ideal for temporary caches.

Question 8

You are configuring Spring Cloud Gateway to sit in front of your Spring Boot microservices. You want to protect a specific `/api/v1/reports` endpoint from abuse.

Your business logic requires that users with a "FREE" subscription tier (extracted from their JWT claim) can only make 10 requests per minute, while users with a "PREMIUM" tier can make 100 requests per minute.

How do you implement this tiered, distributed rate-limiting strategy efficiently at the API Gateway level?

- **A)** Use Spring Security's `OncePerRequestFilter`. Extract the tier from the JWT and use `Thread.sleep()` to artificially delay the HTTP request processing by 6 seconds for FREE users and 0.6 seconds for PREMIUM users, naturally throttling the ingress rate.
- **B)** Configure the built-in Spring Cloud Gateway `RequestRateLimiter` filter factory backed by Redis. Implement a custom `KeyResolver` bean that parses the JWT, extracts the subscription tier, and returns a unique cache key based on the user's ID and tier to apply the appropriate Redis bucket replenishment rate.
- **C)** Install the Netflix Eureka Discovery Client and use its `@LoadBalanced` annotation to route FREE users to a cluster of application servers running on slow hardware, while routing PREMIUM users to servers with higher CPU allocations.
- **D)** Add an `@Aspect` to the downstream microservice controllers that increments a counter in a local static `ConcurrentHashMap`. If the counter exceeds the limit, throw a `ResponseStatusException(HttpStatus.TOO_MANY_REQUESTS)`.

Answer: B

Explanation: Rate limiting should be handled at the Gateway (edge), not in the application instances (which defeats the purpose of protecting them) or via thread sleeping (which exhausts connections). Spring Cloud Gateway integrates natively with Redis to perform highly efficient, distributed token-bucket rate limiting. By writing a custom `KeyResolver`, you tell the Gateway exactly how to identify the user (via JWT) and apply different limits dynamically based on their claims.

Question 9

You are refactoring a Spring Data JPA repository method to fix an N+1 problem. You successfully change the query to: `@Query("SELECT d FROM Department d JOIN FETCH d.employees")`.

Later, a new requirement dictates that this endpoint must be paginated. You update the method signature to accept a `Pageable` object: `Page<Department> findAll(Pageable pageable)`.

When you test the endpoint with `size=10`, the correct 10 departments are returned. However, you notice a massive spike in memory usage, and the Hibernate logs show a severe warning: `HHH000104: firstResult/maxResults specified with collection fetch; applying in memory!`.

What is happening under the hood, and how do you achieve both pagination and eager fetching without crashing the JVM?

- **A)** Hibernate is trying to cast the `Page` object into a `List` dynamically, causing memory bloat. The fix is to change the return type to `Slice<Department>` instead of `Page<Department>`.
- **B)** When combining pagination (`LIMIT/OFFSET`) with `JOIN FETCH` on a collection, the database returns duplicated root entities for every child row. Because Hibernate cannot safely apply the `LIMIT` at the database level without cutting off child records, it fetches the *entire* table into JVM memory and does the pagination in Java. The fix is to remove `JOIN FETCH` and annotate the `employees` collection with `@BatchSize`.
- **C)** The `Pageable` object is incompatible with `@Query` annotations. You must remove the `@Query` annotation and rely strictly on Spring Data's derived query methods (e.g., `findAllOrderByAsc(Pageable pageable)`) to let the framework inject the pagination correctly.
- **D)** The database dialect is incorrectly configured in the `application.yml`. Hibernate is falling back to a legacy ANSI SQL mode that does not support the `LIMIT` keyword. Update the dialect to `org.hibernate.dialect.MySQL8Dialect`.

Answer: B

Explanation: This is one of the most dangerous and common traps in Hibernate. You *cannot* paginate a query that uses JOIN FETCH on a OneToMany collection at the database level. If Department A has 50 employees, a SQL JOIN returns 50 rows for Department A. If you apply a SQL LIMIT 10, you get 1 Department with only 10 of its employees, corrupting the data. To prevent this, Hibernate ignores the SQL LIMIT, downloads the *entire* database into RAM, groups the objects, and *then* hands you the first 10. To fix this, you must query the root entities with pagination first, and use @BatchSize (or a Subselect) to fetch the collections efficiently in a second query.

Question 10

You are writing a unit test for your JPA layer using Spring Boot's @DataJpaTest annotation.

You have a User entity. In the database schema, there is a native database trigger that automatically sets the updated_at timestamp whenever a row is modified.

In your test, you write:

Java

```
User user = userRepository.findById(1L);
```

```
user.setLastName("Smith");
```

```
userRepository.save(user);
```

```
User updatedUser = userRepository.findById(1L);
```

```
assertNotNull(updatedUser.getUpdatedAt()); // This assertion fails
```

You verify that the database trigger works perfectly in production. Why does the updated_at field return null in the test environment, and how do you fix the test?

- **A)** @DataJpaTest uses an embedded H2 database by default, which does not support native database triggers. You must annotate the test class with @AutoConfigureTestDatabase(replace = Replace.NONE) to force the test to run against a real database instance like PostgreSQL.
- **B)** @DataJpaTest executes each test within a transaction and rolls it back at the end. Because the transaction hasn't committed when the second findById is called, the database trigger hasn't fired yet. You must add @Commit to the test method.
- **C)** When you call save(), the entity is updated in Hibernate's First-Level Cache (Persistence Context), but the UPDATE SQL is not immediately sent to the database. The second findById just returns the cached object from memory without hitting the DB. To fix it, you must call entityManager.flush() followed by entityManager.clear() before the second findById.
- **D)** Spring Data JPA's save() method returns a detached proxy object. The variable updatedUser is pointing to the old memory reference from before the save occurred. You must reassign the variable: updatedUser = userRepository.save(user);

Answer: C

Explanation: This tests the core lifecycle of the JPA First-Level Cache. Inside a transaction (which `@DataJpaTest` provides automatically), `save()` does not immediately execute an UPDATE statement; it just queues it. Furthermore, `findById` will always check the local memory cache before hitting the database. Because the object is already in the cache, Hibernate returns it immediately, totally bypassing the database where the trigger lives. To test database-generated values, you must force Hibernate to execute the SQL (`flush()`) and wipe the memory cache (`clear()`) so the next `findById` is forced to fetch fresh data from the physical database.

Question 11

You are building a user registration flow. When a user registers successfully, the `UserService` calls a `NotificationService.sendWelcomeEmail()` method. Because sending the email takes 2 seconds and you don't want to block the HTTP response, you annotate the `sendWelcomeEmail` method with `@Async`.

Inside the email method, you need to log the name of the admin who triggered the registration. You call `SecurityContextHolder.getContext().getAuthentication().getName()`.

During testing, the registration API returns an HTTP 200 OK instantly, but the background thread crashes with a `NullPointerException` because the `Authentication` object is null.

Why is the Security Context empty inside the asynchronous method, and what is the standard Spring Security fix?

- **A)** `@Async` methods execute outside the Spring MVC `DispatcherServlet`. Because Spring Security is implemented as a Servlet Filter, it cannot intercept or apply security contexts to methods bypassing the servlet container. You must manually pass the username as a string parameter to the method.
- **B)** The `SecurityContextHolder` stores the authentication object in a `ThreadLocal` variable by default. `ThreadLocal` variables are strictly isolated to a single thread. When `@Async` spawns a new background thread, the new thread starts with a completely empty context. The fix is to configure the context strategy to `MODE_INHERITABLETHREADLOCAL` or use a `DelegatingSecurityContextExecutor`.
- **C)** By the time the background thread executes, the Tomcat worker thread has already finished processing the HTTP request and has explicitly cleared the `SecurityContext` from application memory to prevent data leaks, destroying the object before the background thread can read it.
- **D)** The `NotificationService` is missing the `@PreAuthorize` annotation. Without a method-level security annotation, Spring's AOP proxy assumes the method is public and intentionally clears the security context to save memory before executing the background task.

Answer: B

Explanation: `ThreadLocal` is the mechanism Spring uses to keep track of "who is logged in" for a specific HTTP request thread. When you use `@Async`, Spring offloads the work to a completely different thread from a Thread Pool. Standard `ThreadLocal` variables do not automatically copy themselves to child or worker threads. To fix this, you must explicitly tell Spring Security to wrap your async executor (using `DelegatingSecurityContextExecutor`) so it knows to copy the security context from the main HTTP thread to the background worker thread before executing the task.

Question 12

You are using Micrometer (MeterRegistry) to instrument a Spring Boot microservice. You want to track the latency of a critical third-party API call.

You create a Timer and add tags (dimensions) to it so you can filter the metrics in your Grafana dashboard. You tag the metric with the http_status code and the user_id of the customer making the request:

```
Timer.builder("api.latency").tag("status", status).tag("user_id",  
userId).register(registry).record(duration);
```

After deploying to production, the service runs perfectly for two days. On the third day, under heavy load, the JVM crashes with an OutOfMemoryError: Java heap space.

What monitoring anti-pattern caused this memory leak, and how should you redesign the metrics?

- **A)** Micrometer stores every individual .record() execution in a bounded in-memory queue. If the Prometheus scraper is configured to pull data every 60 seconds, but your API receives 10,000 requests per second, the queue fills up and crashes the JVM. You must decrease the scrape interval to 5 seconds.
- **B)** Including a highly unique value like user_id creates "High Cardinality". Micrometer creates a brand-new, permanent Timer object in memory for every unique combination of tags. If you have 500,000 users, you create 500,000 distinct Timer objects that the Garbage Collector is never allowed to clean up. You must remove the user_id tag and use distributed tracing (like Zipkin/Jaeger) for user-level tracking.
- **C)** The Timer.builder() method utilizes the Builder pattern, which creates intermediate, short-lived proxy objects. Under heavy concurrent load, the Eden space of the JVM heap becomes fragmented, causing an OOM error before the Minor GC can run.
- **D)** Tagging the HTTP status code as a String forces Micrometer to perform expensive regex parsing internally on every API call. The constant string manipulation exhausts the JVM's String Constant Pool. You must pass the status code as a primitive int.

Answer: B

Explanation: This is the most dangerous trap when implementing application metrics. Time-series databases and metric registries (like Prometheus/Micrometer) are designed to track aggregate trends, not individual events. Every unique combination of tags generates a permanent time-series object in RAM. If you tag a metric with a bounded set (like status: 200, 404, 500), you get 3 objects. If you tag it with user_id (an unbounded set), you get millions of objects stored permanently in memory, inevitably causing an OOM crash.

Question 13

You are processing a massive List<Transaction> containing 5 million records. You need to find a single transaction flagged as "fraudulent" to trigger an immediate system alert.

You write the following Core Java code:

```
Transaction fraud =  
transactions.parallelStream().filter(Transaction::isFraud).findFirst().orElse(null);
```

You notice that even though you are using a parallelStream(), the processing is surprisingly slow, and CPU profiling shows the threads are spending a massive amount of time waiting and coordinating with each other rather than processing the data.

Which Core Java Stream API characteristic is causing this bottleneck, and what is the one-word method change required to optimize it?

- **A)** The `.filter()` operation is inherently stateful and cannot be parallelized. It forces the ForkJoin pool to synchronize the memory boundaries of the list. Change `.filter()` to `.map()`.
- **B)** The `.findFirst()` terminal operation enforces a strict encounter order constraint. Even if Thread 5 finds a fraudulent transaction instantly, it is forced to wait for Threads 1, 2, 3, and 4 to finish processing their chunks to guarantee that Thread 5's finding is actually the *first* one in the original list order. Change `.findFirst()` to `.findAny()`.
- **C)** `parallelStream()` attempts to spawn a new OS-level thread for every single element in the list. Creating 5 million threads immediately exhausts the CPU context switcher. Change `.parallelStream()` to `.stream()` to rely on the default JVM thread.
- **D)** The `Transaction::isFraud` method reference is compiled dynamically via `invokedynamic`, which prevents the JIT compiler from inlining the method across multiple CPU cores. Change the method reference to an explicit lambda: `t -> t.isFraud()`.

Answer: B

Explanation: In a parallel stream, multiple threads process different chunks of the data simultaneously. If you use `findFirst()`, you are telling the JVM: "I don't just want any match, I specifically want the match that appears earliest in the original list." This forces the threads to heavily coordinate. If Thread 8 finds a match, it cannot return it until Threads 1-7 finish and confirm they didn't find one earlier. If you only care about finding *a* match to trigger an alert, use `findAny()`. This removes the ordering constraint, allowing the very first thread that finds a match to immediately terminate the entire stream and return, massively boosting performance.

Question 14

You are working on a complex data import service. You have a public method `processImportFile()`. Inside this method, you parse a CSV file and loop through its lines. For each line, you call a private helper method:

Java

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
```

```
private void saveRecord(RecordData data) {
```

```
    recordRepository.save(data);
```

```
}
```

If a specific record throws a `DataIntegrityViolationException`, you catch it in the loop and expect the code to continue, rolling back only that specific record while committing the rest.

During testing, you discover that if one record fails, none of the `REQUIRES_NEW` transactions work, and the entire file import is either saved or rolled back as a single massive transaction.

Why is the Spring framework completely ignoring the `@Transactional(propagation = Propagation.REQUIRES_NEW)` annotation on your helper method?

- **A)** The `@Transactional` annotation only triggers a new transaction if the underlying database driver supports XA (Two-Phase Commit) protocols. Because you are using a standard JDBC driver, Spring silently falls back to `Propagation.REQUIRED`.

- **B)** Checked exceptions (like `DataIntegrityViolationException`) do not trigger rollbacks in Spring by default. You must add `rollbackFor = Exception.class` to the annotation for it to be recognized.
- **C)** Spring's declarative transaction management relies on dynamic AOP proxies. The proxy wraps the bean and intercepts calls coming from *outside* the bean. Because `processImportFile()` calls `saveRecord()` internally using the `this` reference, the call bypasses the proxy entirely, and the annotation is strictly ignored.
- **D)** Private methods are compiled without metadata parameters in standard Java byte code. Therefore, the Spring component scanner cannot read the annotations via Reflection during the application startup phase.

Answer: C

Explanation: This is the most infamous pitfall of Spring AOP (Aspect-Oriented Programming). Annotations like `@Transactional`, `@Async`, and `@Cacheable` are implemented using proxies. If Bean A calls Bean B, it hits the proxy, and the magic happens. But if a method *inside* Bean B calls another method *inside* Bean B, it is calling the internal implementation directly, completely bypassing the proxy layer. Furthermore, Spring AOP proxies only intercept public methods by default. A private method annotated with `@Transactional` is essentially dead code.

Question 15

You are using `CompletableFuture` to build an asynchronous data aggregation pipeline.

Java

```
CompletableFuture<String> userFuture = CompletableFuture.supplyAsync(() ->
fetchUserData());
```

```
CompletableFuture<String> accountFuture = CompletableFuture.supplyAsync(() ->
fetchAccountData());
```

```
userFuture.thenCombine(accountFuture, (user, account) -> user + " : " + account)
    .thenAccept(result -> System.out.println("Result: " + result));
```

In production, the external API called by `fetchUserData()` occasionally times out and throws a `RuntimeException`. When this happens, the application does not crash, nothing is printed to the console, and the thread simply vanishes silently without executing the rest of the pipeline or logging an error.

Why is the exception vanishing silently, and what is the idiomatic way to handle failures inside a `CompletableFuture` chain?

- **A)** `CompletableFuture` runs on daemon threads provided by the `ForkJoinPool`. Daemon threads do not write their `System.err` streams to the main console, so the stack trace is routed to `/dev/null`. You must provide a custom `ThreadPoolTaskExecutor`.
- **B)** When a step in a `CompletableFuture` chain throws an exception, the future transitions to a "completed exceptionally" state. If you do not explicitly attach an error-handling callback to the chain, the exception is swallowed and downstream operations (like `thenCombine` or `thenAccept`) are simply skipped silently. You must append `.exceptionally(ex -> { log(ex); return "Fallback"; })` to the chain.

- **C)** The `thenCombine` operator requires both futures to complete successfully. Because one failed, it triggers a `TimeoutException` which is a checked exception. The Java compiler suppresses checked exceptions in lambda expressions by default.
- **D)** The `RuntimeException` is being caught by the global `@RestControllerAdvice` in Spring MVC, which intercepts all unhandled exceptions and converts them to HTTP 500 responses before the `System.out.println` can execute.

Answer: B

Explanation: By design, the `CompletableFuture` API does not crash your main thread when a background task throws an exception (unless you call `.join()` or `.get()` on the failed future). Instead, the future simply marks its internal state as "exceptional". Any subsequent functional steps (`thenApply`, `thenAccept`) attached to that future see the exceptional state and refuse to run, silently halting the pipeline. To handle this, you must explicitly use error-handling operators like `.exceptionally()`, `.handle()`, or `.whenComplete()` to catch the error, log it, and potentially return a default fallback value so the pipeline can continue.

Question 16

You are tasked with building a global auditing feature for a Spring Boot REST API. The requirement is to log the exact JSON payload of every HTTP request and response for security compliance. A developer implements a `HandlerInterceptor` and overrides the `afterCompletion` method to read the `HttpServletResponse` output stream and log the body.

However, in production, the application starts throwing `IllegalStateException`: `getOutputStream()` has already been called for this response, and the audit logs show completely empty response bodies.

What is the architectural reason for this failure, and what is the standard Spring ecosystem pattern to resolve it?

- **A)** Interceptors are executed asynchronously in a separate thread. By the time `afterCompletion` runs, the Tomcat worker thread has already closed the HTTP socket. The fix is to use `@Async` on the interceptor method.
- **B)** The HTTP response stream is a forward-only data stream. Once the Spring MVC `HttpMessageConverter` writes the JSON to the response stream and flushes it to the client, the stream is consumed and cannot be read again. The fix is to implement a `Filter` that wraps the request/response in `ContentCachingRequestWrapper` and `ContentCachingResponseWrapper`.
- **C)** `HandlerInterceptor` is strictly designed for view-based applications (Thymeleaf/JSP). For `@RestController` applications, you must use a `@ControllerAdvice` implementing `ResponseBodyAdvice` to intercept the JSON before serialization.
- **D)** The application is likely using gzip compression (`server.compression.enabled=true`). The interceptor is trying to read raw bytes that have already been compressed, causing an EOF exception masked as an `IllegalStateException`. The fix is to disable gzip.

Answer: B

Explanation: This is a classic stream consumption problem in web development. HTTP request and response bodies are represented as `InputStream` and `OutputStream`. You can only read or write to a stream once. If the application writes the response to the client, you cannot read it again in an interceptor to log it. Spring provides `ContentCachingResponseWrapper` (used inside a `Servlet Filter`, which sits higher in the chain than an `Interceptor`). It wraps the stream, intercepts the bytes as they are written

to the client, caches them in memory, and allows you to read the cached bytes for logging after the request completes (remembering to call `copyBodyToResponse()`).

Question 17

You are developing an internal open-source Spring Boot library (company-s3-starter) to provide a standardized `StorageService`. You use `@ConditionalOnMissingBean` on your library's `StorageService` bean so that any microservice using your library can easily override it with their own implementation if needed.

You instruct developers to add your library as a Maven dependency. A developer adds your library, creates their own custom `StorageService` bean annotated with `@Service` in their main application package, and starts the app.

The application crashes immediately because your library's default `StorageService` attempts to load, overriding the developer's custom bean, and fails due to missing default AWS credentials.

Why did `@ConditionalOnMissingBean` fail to back down, and how must you package your starter library to fix it?

- **A)** The developer used `@Service` instead of `@Bean` inside a `@Configuration` class. `@ConditionalOnMissingBean` only evaluates beans explicitly defined in configuration classes, not those discovered via component scanning.
- **B)** Your library's configuration class was likely placed in a package that is automatically scanned by the developer's `@SpringBootApplication`. Because component scanning happens *before* auto-configuration, your bean was evaluated before the developer's custom bean was registered. Your config class must be registered in the `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` file and placed outside the main component scan path.
- **C)** You must explicitly specify the bean name in the condition: `@ConditionalOnMissingBean(name = "customStorageService")`. Without the name, Spring Boot evaluates the condition based on the interface, which causes a circular dependency resolution failure.
- **D)** `@ConditionalOnMissingBean` only works if the overriding bean is marked with `@Primary`. You must update the library documentation to instruct developers to add `@Primary` to their custom implementations.

Answer: B

Explanation: This tests a deep understanding of the Spring Boot application lifecycle. User-defined beans (found via `@ComponentScan`) are processed first. Auto-configuration classes are processed *last*. `@ConditionalOnMissingBean` relies on this exact order: it checks if the user has defined the bean, and if not, it provides the default. If an auto-configuration class is accidentally picked up by the user's `@ComponentScan`, it is evaluated immediately alongside the user's beans. The condition checks the context, sees no bean *yet*, registers the default, and then crashes when it collides with the user's bean later. Auto-configurations must be loaded via the `AutoConfiguration.imports` file, not component scanning.

Question 18

You are building a high-throughput video processing service. When a user uploads a video, you dispatch a background task to generate a thumbnail. You use Core Java's `ExecutorService`:

Java

```
ExecutorService executor = Executors.newFixedThreadPool(10);

public void processVideo(File video) {

    executor.submit(() -> {

        generateThumbnail(video); // Might throw CorruptedFileException (RuntimeException)

    });

}
```

In production, several corrupted videos are uploaded. The generateThumbnail method correctly throws a CorruptedFileException. However, you notice that these exceptions are completely swallowed—they do not crash the application, but they also never appear in the application logs, your APM tools, or the console.

Why is the ExecutorService swallowing the exception, and what is the standard Core Java fix for "fire-and-forget" background tasks?

- **A)** Uncaught exceptions in thread pools are automatically routed to the JVM's System.err stream, which Docker and Kubernetes do not capture by default. You must redirect System.err to System.out at application startup.
- **B)** The submit() method accepts a Callable or Runnable and returns a Future. If an exception occurs, the executor catches it and stores it inside the Future object. If you never call future.get(), the exception is permanently hidden. To fix it for fire-and-forget tasks, use executor.execute() instead, which will throw the exception to the thread's UncaughtExceptionHandler.
- **C)** Thread pools intercept RuntimeException classes to prevent the worker thread from dying. You must change CorruptedFileException to extend java.lang.Exception (a checked exception) so the JVM is forced to print the stack trace.
- **D)** The fixed thread pool size of 10 creates a bounded queue. When an exception is thrown, the thread pool immediately clears the queue to prevent memory leaks, deleting the stack trace from memory before the logging framework can access it.

Answer: B

Explanation: This is a crucial distinction in the ExecutorService API. If you use execute(Runnable), the code runs, and if it throws an unhandled exception, the thread dies, the exception bubbles up to the UncaughtExceptionHandler, and it is usually logged. If you use submit(Runnable/Callable), the executor assumes you are going to check the results later. It catches any exception and safely packages it inside the returned Future object. If you never call future.get() (which is common in "fire-and-forget" tasks), the exception is swallowed entirely, leading to silent failures.

Question 19

You are tasked with implementing a "Delete Account" feature. You have a Company entity with a bidirectional @OneToMany relationship to Employee. The requirement is that when a company is deleted, all of its employees must also be deleted.

You set cascade = CascadeType.ALL and orphanRemoval = true on the employees collection inside the Company entity.

A company with 25,000 employees is deleted using `companyRepository.delete(company)`. The database grinds to an absolute halt, causing a 30-second system-wide outage. The DBA sends you the query logs, showing exactly 25,001 separate SQL DELETE statements executed sequentially.

Why does Spring Data JPA execute this massive N+1 deletion anomaly, and what is the correct, performant way to delete the hierarchy?

- **A)** `CascadeType.ALL` forces JPA to use Optimistic Locking during deletions. It executes individual deletes to verify the `@Version` column of each employee to prevent concurrent modification exceptions. The fix is to change the cascade type to `CascadeType.REMOVE`.
- **B)** JPA handles cascade deletes at the application level, not the database level. To trigger potential `@PreRemove` lifecycle callbacks on every single child entity, Hibernate pulls all 25,000 employees into JVM memory and executes a separate DELETE for each one. The performant fix is to write a custom `@Modifying` query: `@Query("DELETE FROM Employee e WHERE e.company = :company")` and call it explicitly before deleting the company.
- **C)** The database schema is missing a foreign key constraint with ON DELETE CASCADE. If the constraint is added to the SQL schema, Hibernate will automatically detect it on startup and switch to executing a single DELETE FROM company statement, letting the database handle the children.
- **D)** The `delete()` method in Spring Data JPA is transactional but not batched. You must add the property `spring.jpa.properties.hibernate.jdbc.batch_size=1000` to the YAML file to force Hibernate to bundle the 25,000 deletes into 25 network calls instead of 25,000.

Answer: B

Explanation: This is one of the most dangerous performance traps in JPA. When you rely on `CascadeType.REMOVE` (or `ALL`), Hibernate does *not* issue a bulk DELETE FROM employee WHERE company_id = ?. Because JPA must respect application-level lifecycle events (like `@PreRemove` or `EntityListeners`), it must load every child entity into memory and delete them one-by-one. For massive collections, this brings down the database and the JVM. To perform bulk deletes efficiently, you must bypass the JPA cascade mechanism and execute explicit bulk `@Modifying` JPQL/SQL queries. (Option D helps network latency slightly, but still requires pulling 25k records into memory first).

Question 20

You are building a Spring Boot application that consists of a React frontend and a REST API backend. You want users to log in using their Google accounts.

You configure the backend with `spring-boot-starter-oauth2-client` and write the following security config: `http.oauth2Login()`.

When the user clicks "Login", they are redirected to Google, authenticate successfully, and are redirected back to the React app. The React app extracts the Google JWT from the URL/response and attaches it as a Bearer token in the Authorization header for subsequent API calls (e.g., GET /api/profile).

However, the Spring Boot API rejects every request with a 401 Unauthorized.

What fundamental OAuth2 architectural concept is misconfigured in the Spring Boot backend?

- **A)** The Google JWT is signed using Google's private RSA keys. Spring Boot cannot validate the token because it does not have access to Google's public JWKS (JSON Web Key Set) endpoint. You must manually configure the `spring.security.oauth2.resourceserver.jwt.issuer-uri` property.
- **B)** `oauth2Login()` configures the application as an OAuth2 *Client*. It is designed to handle the authorization code flow, create a stateful server-side session (JSESSIONID), and rely on cookies. It does not configure the application to validate incoming Bearer tokens. To accept Bearer tokens from a frontend, the backend must be configured as an OAuth2 *Resource Server* using `http.oauth2ResourceServer().jwt()`.
- **C)** Cross-Origin Resource Sharing (CORS) drops the Authorization header by default during the preflight request. You must explicitly expose the Authorization header in your `CorsConfiguration` bean using `.addExposedHeader("Authorization")`.
- **D)** Google issues OIDC (OpenID Connect) tokens, not standard OAuth2 JWTs. Spring Security's default `oauth2Login` filter expects a generic JWT structure and fails to parse the specific `iss` and `aud` claims provided by Google. You must write a custom `JwtDecoder`.

Answer: B

Explanation: This highlights the crucial distinction between an OAuth2 Client and an OAuth2 Resource Server. `oauth2Login()` uses the Authorization Code flow (typically for monolithic, server-rendered apps like Thymeleaf) and establishes a stateful cookie-based session. It expects the browser to send a cookie, not a Bearer token. A REST API acting as a stateless backend for a React/Angular app must act as a Resource Server. The frontend is responsible for obtaining the token (acting as a public client), and the backend is only responsible for validating the incoming Bearer token using `oauth2ResourceServer().jwt()`.

Question 21

You implement a custom Spring AOP aspect annotated with `@Around("@annotation(LogExecutionTime)")`. This aspect starts a stopwatch, calls `joinPoint.proceed()`, stops the stopwatch, and logs the total execution time of the method.

You place your `@LogExecutionTime` annotation on a complex service method that is also annotated with `@Transactional`.

During a performance audit, the DBA proves that the database transaction for this method consistently takes 4.5 seconds to execute (mostly blocked by a table lock during the commit phase). However, your custom AOP log output consistently reports an execution time of only 0.2 seconds.

Why is the execution time drastically underreported, and how do you configure the architecture to measure the true transaction duration?

- **A)** The custom aspect is executing asynchronously on a separate thread, causing the stopwatch to evaluate immediately before the underlying database driver has finished network transmission. You must remove `@Async` from your aspect configuration.
- **B)** By default, Spring's `@Transactional` proxy has a higher priority (lower order) than custom aspects. Therefore, the custom aspect wraps the *target object inside* the transaction proxy. The stopwatch only measures the Java method execution, completely missing the database flush and commit phases handled by the outer transaction proxy. The fix is to add `@Order(Ordered.HIGHEST_PRECEDENCE)` to your custom aspect class.
- **C)** The AOP proxy uses bytecode instrumentation. When measuring methods that return void, the stopwatch terminates as soon as the method frame is popped from the JVM stack, ignoring background Hibernate batching. You must change the method to return the saved entity.

- **D)** @Transactional and custom @Around advice are mutually exclusive on the same method. Spring silently ignores the @Around advice during the commit phase to prevent memory leaks. You must implement a TransactionSynchronization callback instead.

Answer: B

Explanation: Spring AOP applies cross-cutting concerns using a chain of proxies. If multiple proxies wrap a method, their execution order matters massively. By default, Spring's transaction infrastructure has a very low @Order value (meaning it executes first, forming the outermost proxy). Your custom aspect forms an inner proxy. It starts the timer, runs the Java code, and stops the timer. Then it returns to the transaction proxy, which *then* flushes the session and commits to the database (where the 4.5-second delay happens). To measure the commit time, your custom aspect must wrap the transaction proxy by giving it a higher precedence (lower order number).

Question 22

Your team uses Testcontainers for integration testing to spin up a real PostgreSQL Docker container. You write two test classes: OrderServiceTest and PaymentServiceTest.

Both classes are annotated with @SpringBootTest and use JUnit 5's @Container annotation to define the PostgreSQL instance.

When you run the test suite locally, OrderServiceTest executes and passes flawlessly. Immediately after, PaymentServiceTest starts, but every single database query inside it fails with a org.postgresql.util.PSQLException: Connection to localhost:XXXXX refused.

What Spring Boot testing lifecycle mechanism is clashing with Testcontainers to cause this connection failure on the second test class?

- **A)** Spring Boot's internal HikariCP connection pool caches the database credentials statically. The second test class generates a new random password for the Docker container, but HikariCP attempts to use the password from the first test class.
- **B)** Spring's TestContext framework aggressively caches the ApplicationContext (including the DataSource bean) across multiple test classes if their configuration is identical. However, the @Container annotation shuts down and destroys the Docker container after the first test class finishes. The second test reuses the cached Spring context, which is now pointing to a dead Docker port.
- **C)** The Docker daemon limits the total number of bound TCP ports per JVM process. The second test class attempts to bind to the same ephemeral port that is currently stuck in a TIME_WAIT state from the first test container.
- **D)** The @SpringBootTest annotation automatically executes Flyway/Liquibase migrations. On the second test class, the migration scripts attempt to create tables that already exist, throwing a fatal startup error that masks the connection exception.

Answer: B

Explanation: This is the most common Testcontainers pitfall. Spring Boot tries to be fast by creating one Application Context and sharing it across all test classes. JUnit 5's @Container (when used as an instance or static field) ties the container's lifecycle to the test class—it destroys the container after the class finishes. When the second class runs, Spring says, "I already have a context, here it is!" But the DataSource inside that context has a JDBC URL pointing to port 54321, which Docker just killed. The standard fix is to use the "Singleton Container Pattern" (a static container block without the @Container annotation) so the database stays alive for the entire JVM run.

Question 23

You are developing a dynamic pricing engine in Spring Boot. To allow business analysts to change pricing rules without redeploying the app, you integrate a Groovy scripting engine. The analysts upload new Groovy scripts via a REST API, and your application compiles and executes these scripts on the fly using the JVM's ScriptEngine.

The feature works great, but after a week of heavy usage and frequent script updates, the application crashes with a fatal error: `java.lang.OutOfMemoryError: Metaspace`. You check the standard JVM heap, and it is less than 30% full.

What specific JVM memory component have you exhausted, and why is this dynamic architecture causing the leak?

- **A)** Metaspace holds the JVM's thread stacks. Executing external scripts causes the application to spawn runaway background threads that never terminate, exhausting the stack allocation.
- **B)** Groovy scripts are interpreted, not compiled. The interpreter caches the raw String representation of every executed script in the Metaspace. You must call `.intern()` on the scripts before executing them.
- **C)** Metaspace is the native memory region where the JVM stores Class metadata (method bytecode, static variables). Every time a new script is executed, the scripting engine compiles it into a brand new Java Class. Because the old classes are still referenced by the engine's internal classloader, the Garbage Collector cannot unload them, permanently filling the Metaspace.
- **D)** Metaspace is the designated memory area for Direct Byte Buffers (NIO). The scripting engine reads the uploaded files using NIO channels but fails to close them, leaking native OS memory.

Answer: C

Explanation: Before Java 8, this area was called the "PermGen"; now it is "Metaspace". It does not store objects (like the Heap does); it stores the definitions of Classes themselves. When you dynamically compile code (Groovy, Nashorn, or heavily using CGLIB proxies), the JVM creates new Class definitions in Metaspace. If you generate thousands of dynamic classes and don't strictly manage their ClassLoaders so they can be garbage collected, the Metaspace fills up, causing a catastrophic JVM crash regardless of how much standard Heap memory is available.

Question 24

You are building an Admin Dashboard. A frontend React app fetches a user profile via `GET /api/users/1`. The API maps the User entity to a UserDTO and returns it.

The User entity has an `@Version Long version`; field to enable JPA Optimistic Locking. The version field is correctly included in the UserDTO sent to the frontend.

Admin A and Admin B load the profile simultaneously (both receive version = 1).

Admin A changes the name and clicks save. The frontend sends a `PUT /api/users/1` with version = 1. Your service does the following:

Java

```
User user = userRepository.findById(dto.getId());
```

```
user.setName(dto.getName());
```

```
userRepository.save(user);
```

Admin B then changes the email and clicks save. The frontend sends a PUT /api/users/1 with version = 1. Your service executes the exact same code.

Instead of throwing an `ObjectOptimisticLockingFailureException` on Admin B's request, the update succeeds, and Admin A's name change is permanently lost.

Why did Optimistic Locking fail to protect the data in this disconnected state?

- **A)** Optimistic Locking only works natively on `@Transactional(isolation = Isolation.REPEATABLE_READ)`. Because the default is `READ_COMMITTED`, the database allowed the concurrent transaction to overwrite the row.
- **B)** When the service called `findById()` for Admin B, Hibernate loaded the *new* row (which Admin A just updated) into memory, including its incremented version = 2. Because you mapped the new name but failed to map the version = 1 from Admin B's DTO back onto the loaded entity before saving, Hibernate assumed Admin B was working with the freshest data and allowed the save.
- **C)** The `@Version` annotation relies on the First-Level Cache. Because the GET and PUT requests occurred in entirely separate HTTP sessions, the Hibernate Session was cleared, nullifying the lock state.
- **D)** Spring Data's `save()` method automatically disables Optimistic Locking when updating existing records to prioritize availability over consistency. You must use `saveAndFlush()` to enforce the lock check.

Answer: B

Explanation: This is the most common failure point of "Disconnected" Optimistic Locking. When handling DTOs, the framework is totally blind to what version the user *was* looking at unless you tell it. When Admin B makes the PUT request, your code does a fresh `findById()`. This pulls the latest data (version 2) into the JPA entity. You update the name, but you leave the version as 2. Hibernate compares the entity's version (2) with the database (2), sees they match, increments it to 3, and saves it. To make optimistic locking work with DTOs, you *must* manually set the entity's version to the DTO's version (`user.setVersion(dto.getVersion())`) before calling `save()`. Then Hibernate will compare 1 against 2, and throw the exception.

Question 25

You are developing a batch processing microservice. The service has a HikariCP database connection pool configured with exactly 10 maximum connections.

You write a service method that initiates a transaction to insert a parent Company record. Then, for performance, you attempt to insert its 50 Employee records in parallel using Java's `CompletableFuture`:

```
Java
```

```
@Transactional
```

```
public void saveCompanyData(Company company, List<Employee> employees) {
```

```
    companyRepository.save(company); // Step 1
```

```
// Step 2: Save employees in parallel
```

```
List<CompletableFuture<Void>> futures = employees.stream()
```

```
.map(emp -> CompletableFuture.runAsync(() -> employeeRepository.save(emp)))
```

```
.toList();
```

```
CompletableFuture.allOf(futures.toArray(new CompletableFuture[0])).join(); // Step 3
```

```
}
```

When two users hit this endpoint simultaneously, the application completely freezes. No errors are thrown, CPU usage drops to zero, and all incoming API requests immediately time out.

What specific architectural bottleneck has caused this application-level deadlock?

- **A)** `CompletableFuture.runAsync()` uses the `ForkJoin` pool, which has a default timeout of 60 seconds. The `.join()` method blocks the main Tomcat thread indefinitely because the `ForkJoin` threads are stuck waiting for the CPU context switcher.
- **B)** This is Thread Starvation Deadlock caused by connection pool exhaustion. The main thread holds 1 database connection for the `@Transactional` block. It then spawns parallel background threads, each of which requests its own database connection to execute `employeeRepository.save()`. With 2 users, the main threads hold 2 connections, and spawn 100 child threads fighting for the remaining 8 connections. The children cannot get connections, so they wait forever. The main thread waits forever for the children to finish via `.join()`.
- **C)** The `@Transactional` annotation places an exclusive row-level lock on the Company table. The background threads are attempting to insert Employee records that have foreign keys pointing to the locked Company row, causing the database to pause the inserts to prevent referential integrity corruption.
- **D)** Spring Data JPA repositories are not thread-safe. Injecting `employeeRepository` into a parallel stream causes the underlying Hibernate Session to throw a `ConcurrentModificationException` internally, which the `CompletableFuture` silently swallows, leaving the `.join()` method waiting forever.

Answer: B

Explanation: This is a terrifying and common production killer: Connection Pool Starvation. `ThreadLocal` variables (which hold the active database connection for a `@Transactional` block) do *not* propagate to background threads by default. Therefore, every `runAsync` thread asks HikariCP for a *brand new* connection. If your connection pool has 10 connections, and the main thread is holding one while waiting for 50 child threads that also need connections, the pool runs dry instantly. The child threads block waiting for the pool. The main thread blocks waiting for the children. Deadlock. Never perform multithreaded database writes originating from inside an active database transaction.

Question 26

Your application includes a `PaymentService` with a `@Transactional` method. Inside this method, you create a new `PaymentRecord` and call

paymentRepository.save(paymentRecord). On the very next line, you publish a Spring Event containing the ID of the new payment.

An asynchronous @EventListener in a completely separate NotificationService catches this event, extracts the payment ID, and immediately queries the database to load the PaymentRecord to send an email.

However, the NotificationService consistently throws a EntityNotFoundException. You check the database manually after the process completes, and the record exists perfectly.

Why is the NotificationService failing to find the record that was just saved?

- **A)** Spring Events are dispatched via a UDP network layer by default, which can cause race conditions where the event arrives before the Java thread has finished executing the method.
- **B)** paymentRepository.save() only attaches the entity to the Hibernate First-Level Cache and queues the SQL INSERT. The actual flush and commit to the physical database happen at the very end of the @Transactional method. The asynchronous listener queries the database before the parent transaction has committed.
- **C)** The PaymentRecord entity is missing the @Version annotation, which prevents Spring Data from immediately flushing the newly generated primary key to the transaction log.
- **D)** The asynchronous thread utilizes a read-only database replica. Replica synchronization takes up to 5 seconds, causing a "stale read" anomaly. The fix is to add @Transactional(readOnly = false) to the listener.

Answer: B

Explanation: This is a classic transactional race condition. When you call save(), Hibernate does not immediately insert the row; it waits until the end of the transaction to batch operations efficiently. By publishing the event immediately after save(), the asynchronous listener thread starts querying the physical database while the original thread's transaction is still open and uncommitted. The listener finds nothing. The correct architectural fix is to annotate the listener with @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT), guaranteeing the event is only processed *after* the database commit is successful.

Question 27

You are reviewing a junior developer's code. They need to ensure that a critical section of code updating a shared counter is only executed by one thread at a time. They wrote the following implementation:

Java

```
private Boolean isProcessing = false;
```

```
public void processOrder() {
```

```
    synchronized (isProcessing) {
```

```
        isProcessing = true;
```

```
        // ... perform critical business logic ...
```

```
isProcessing = false;  
  
}  
  
}
```

During a load test with 50 concurrent threads, the application experiences massive data corruption because multiple threads are entering the critical section simultaneously.

What fundamental Core Java concurrency principle is being violated, causing the synchronized block to fail completely?

- **A)** The Boolean class in Java is not thread-safe. You must replace it with an AtomicBoolean to use it as a lock monitor.
- **B)** Java's autoboxing creates a brand new Boolean object in memory every time isProcessing is reassigned to true or false. Because the synchronized keyword locks on the *object memory reference*, reassigning the variable changes the reference, allowing the next thread to lock on a completely different object and enter the block.
- **C)** The synchronized keyword only works on methods, not on primitive wrapper classes. The JVM silently ignores the block, executing the code concurrently.
- **D)** The isProcessing variable lacks the volatile keyword, meaning the thread-local CPU caches are not syncing the lock state with main memory.

Answer: B

Explanation: This is a fatal concurrency flaw. synchronized(object) acquires a monitor lock on the specific memory address of that object. In Java, Boolean is an immutable object. When the code executes isProcessing = true, it does not change the internal state of the existing object; autoboxing creates a pointer to a *new* Boolean object in memory. If Thread A locks on Object 1 and then changes the variable to point to Object 2, Thread B comes along, sees the variable points to Object 2, acquires the lock on Object 2, and enters the critical section while Thread A is still inside. You should always synchronize on a final object instance (e.g., private final Object lock = new Object();).

Question 28

Your team decides to migrate a high-throughput microservice from standard Spring MVC to Spring WebFlux (Project Reactor) to handle 10x more traffic.

You rewrite the REST controllers to return Mono and Flux. Inside one of the reactive streams, you map over the data and execute a legacy JDBC query using JdbcTemplate.queryForObject().

When you deploy this to production, traffic hits the service. Initially, it works, but within 10 seconds, the application completely freezes. It consumes almost 0% CPU, but every single incoming HTTP request times out.

What is the architectural cause of this total application freeze in the Spring WebFlux environment?

- **A)** WebFlux runs on an asynchronous Netty server which utilizes a very small number of Event Loop threads (typically one per CPU core). The synchronous JdbcTemplate call blocks the executing thread while waiting for the database network response. By blocking all available Event Loop threads, the server becomes instantly incapable of accepting or processing any new HTTP connections.

- **B)** JdbcTemplate relies on ThreadLocal for transaction management. Because WebFlux constantly switches threads during stream processing, the transaction context is lost, causing the database connection pool to deadlock permanently.
- **C)** Reactive streams (Mono/Flux) are entirely lazy. Because the JdbcTemplate is placed inside a .map() operator without a .subscribe() call on the JDBC result, the query is never actually dispatched to the database.
- **D)** The reactive Backpressure mechanism detects that the database is slower than the incoming HTTP requests. It intentionally drops all TCP connections to protect the database from being overwhelmed.

Answer: A

Explanation: The golden rule of Reactive Programming (WebFlux/Node.js) is "Never block the Event Loop." Standard Spring MVC (Tomcat) uses a "thread-per-request" model; if you have 200 threads, you can block 200 threads waiting for a DB. WebFlux uses an Event Loop model; if you have a 4-core CPU, you might only have 4 worker threads. If you execute a blocking JDBC/I/O call, you permanently freeze 25% of your server's entire capacity. Freeze all 4, and the server dies. To fix this, you must either use a reactive database driver (R2DBC) or wrap the blocking JDBC call in a dedicated, isolated thread pool using `publishOn(Schedulers.boundedElastic())`.

Question 29

Your Spring Boot application acts as an OAuth2 Resource Server. It validates incoming JWT Bearer tokens issued by an external Authorization Server (like Auth0 or Keycloak).

The JWTs are signed using the RS256 algorithm (asymmetric encryption).

A senior developer configures the security chain by hardcoding the Authorization Server's public key directly into the `application.yml` property:
`spring.security.oauth2.resourceserver.jwt.public-key-location.`

The application runs flawlessly for 90 days. On day 91, every single incoming API request suddenly fails with a 401 Unauthorized and a `SignatureException` in the logs, even though the tokens are freshly generated and perfectly valid.

What standard security lifecycle event occurred on the Authorization Server, and what is the proper architectural configuration to prevent this outage?

- **A)** The Authorization Server rotated its cryptographic keypair. Because your Resource Server hardcoded the old public key, it can no longer verify the signatures of tokens minted with the new private key. You must configure the `jwk-set-uri` (JSON Web Key Set) property instead, which allows Spring Security to dynamically fetch and cache the correct public keys based on the token's kid header.
- **B)** RS256 JWTs have a hardcoded maximum absolute lifespan of 90 days built into the Spring Security algorithm registry. You must migrate the Authorization Server to use the ES256 (Elliptic Curve) algorithm.
- **C)** The hardcoded public key was read from the filesystem at startup and stored in the JVM Metaspace. A minor Garbage Collection cycle cleared the key from memory, causing all signature validations to fail. You must define the key as a Spring `@Bean` to prevent GC.
- **D)** The clock skew between your Resource Server and the Authorization Server drifted by more than 5 minutes. The JWT `exp` (expiration) and `nbf` (not before) claims became mathematically invalid. You must install an NTP (Network Time Protocol) daemon on the server.

Answer: A

Explanation: Security best practices mandate that Authorization Servers regularly rotate their cryptographic signing keys to limit the impact of a potential key compromise. If you hardcode a public key in your application, your application will break the moment the Auth Server rotates its keys. The industry standard is JWKS (JSON Web Key Set). By configuring the `jwt-set-uri`, Spring Security automatically inspects incoming JWTs, reads the Key ID (`kid`) from the token header, and dynamically queries the Auth Server's JWKS endpoint to download the matching public key to verify the signature.

Question 30

You implement caching in your Spring Boot application to improve performance. You have a `RoleService` with a method annotated with `@Cacheable`:

Java

```
@Cacheable("roles")
```

```
public List<Role> getAllRoles() {  
    return roleRepository.findAll();  
}
```

Admin A makes a request to a controller that calls `getAllRoles()`. The service hits the database, caches the list, and returns it. Admin A's controller then decides to add a temporary "GUEST" role to that specific returned list for display purposes: `roles.add(new Role("GUEST"));`

Admin B immediately hits a completely different controller that also calls `roleService.getAllRoles()`. Admin B receives the list directly from the cache.

What unexpected data anomaly will Admin B experience, and why?

- **A)** Admin B will receive a `NullPointerException`. The cache serialization mechanism destroys the original `List` object when converting it to a byte stream, leaving the reference null.
- **B)** Admin B will receive the list containing the temporary "GUEST" role added by Admin A. Spring's default `ConcurrentMapCacheManager` stores the actual object *memory reference* in the cache. When Admin A mutated the returned list, they mutated the globally cached instance.
- **C)** Admin B will receive the correct, pristine list from the database. Spring automatically detects modifications to cached objects using AOP and immediately evicts the corrupted key from the cache.
- **D)** Admin B will trigger a `ConcurrentModificationException` because the cache manager locks the collection asynchronously while the second controller attempts to read it.

Answer: B

Explanation: This is a very common and dangerous side effect of using Spring's default, in-memory cache manager without a serialization layer (like Redis). The default cache is literally a `ConcurrentHashMap`. When you cache an object or a list, Spring puts the *pointer* to that object in the map. When you retrieve it, Spring hands you the *pointer*. If the caller modifies the object they receive, they are modifying the exact object sitting inside the cache, corrupting the data for every subsequent user. Cached objects in a local

memory cache must be strictly treated as immutable, or the cache manager must be configured to perform deep copies.

Question 31

You are writing unit tests for a Spring Boot REST API. You create a test class for `UserController` and annotate it with `@WebMvcTest(UserController.class)`.

The `UserController` depends on a `UserService` via constructor injection, which in turn depends on a `UserRepository` (a Spring Data JPA interface).

You want to test the JSON serialization of the controller, so you do not mock the `UserService`; you want the real service to execute, but you use `@MockBean` for the `UserRepository`.

When you run the test, it instantly fails to start, throwing a `NoSuchBeanDefinitionException`: No qualifying bean of type 'UserService' available.

Why did the Spring Test Context fail to find the `UserService` bean, even though it is correctly annotated with `@Service` in your main source code?

- **A)** The `@MockBean` on `UserRepository` creates a circular dependency in the test context, forcing Spring to abort the initialization of the service layer.
- **B)** `@WebMvcTest` is a "sliced" test annotation. It intentionally restricts component scanning to strictly web-layer beans (`@Controller`, `@ControllerAdvice`, `Filter`, etc.) to keep the test lightning-fast. It actively ignores `@Service` and `@Component` annotations. The fix is to add `@Import(UserService.class)` to the test class.
- **C)** When you specify the controller class in the annotation (`@WebMvcTest(UserController.class)`), Spring Boot automatically disables all dependency injection. You must manually instantiate the controller using the new keyword in an `@BeforeEach` setup method.
- **D)** The `UserService` is likely missing a public no-args constructor, which is strictly required by the standard Spring Boot testing framework to proxy the bean before injecting the mocked repository.

Answer: B

Explanation: This tests a fundamental understanding of Spring Boot's "Test Slices." Annotations like `@WebMvcTest` or `@DataJpaTest` do not load the entire application context (like `@SpringBootTest` does). `@WebMvcTest` only loads the MVC infrastructure and your controllers. It completely ignores the service and repository layers. If your controller needs a service, you must either mock that service directly using `@MockBean` inside the test class, or explicitly tell Spring to load the real service using `@Import`.

Question 32

You are optimizing a heavily utilized Spring Boot API. You need to call three slow, third-party REST APIs and aggregate their results. You rewrite the method using Core Java's `CompletableFuture`:

Java

```
CompletableFuture<String> res1 = CompletableFuture.supplyAsync(() ->
slowApiClient.get("api1"));
```

```
CompletableFuture<String> res2 = CompletableFuture.supplyAsync() ->  
slowApiClient.get("api2"));
```

```
// ... aggregate and return
```

Under normal traffic, response times drop from 9 seconds to 3 seconds. However, during a traffic spike, the entire Tomcat server locks up. Thread dumps reveal that all HTTP requests are backed up, and the JVM's `ForkJoinPool.commonPool()` has completely exhausted its threads, which are all stuck in a `WAITING` state on network I/O.

What Core Java concurrency mistake caused this lockup, and how should you architect the `CompletableFuture` calls for I/O-bound tasks?

- **A)** `CompletableFuture` is designed exclusively for CPU-bound mathematical computations, not network calls. You must replace it with Spring's `@Async` annotation, which natively utilizes non-blocking Java NIO sockets.
- **B)** The `supplyAsync()` method without an explicit `Executor` defaults to the `ForkJoinPool.commonPool()`. This pool is sized exactly to the number of CPU cores minus one. If you have a 4-core server, you only have 3 threads. Blocking these 3 threads with slow network I/O instantly starves the entire JVM of background processing power. The fix is to pass a custom, properly sized `ThreadPoolExecutor` as the second argument to `supplyAsync()`.
- **C)** The network TCP connections are not being explicitly closed inside the lambda expressions, causing a memory leak in the `ForkJoin` pool's internal task queue. The fix is to wrap the API calls in a `try-with-resources` block.
- **D)** The threads are deadlocking because `CompletableFuture` requires you to call `.join()` on them in the exact same order they were dispatched. Calling them out of order corrupts the thread scheduler's synchronization barriers.

Answer: B

Explanation: This is a devastating and common production flaw. The `commonPool()` is designed for short, non-blocking, CPU-intensive tasks (like parallel streams). Its size is very small (usually equal to your CPU cores). If you use it for I/O bound tasks (like HTTP calls or DB queries), those few threads get stuck waiting for the network. Because the common pool is shared globally across the entire JVM, starving it can break other completely unrelated parts of your application. You *must* always provide a custom thread pool (e.g., `Executors.newFixedThreadPool(50)`) when using `CompletableFuture` for blocking I/O operations.

Question 33

You are integrating Spring Security as an OAuth2 Resource Server to validate JWTs.

The Authorization Server issues JWTs containing a "scope" claim: "scope": "read write admin".

In your controller, you secure an endpoint using `@PreAuthorize("hasRole('ADMIN')")`.

Even though the JWT is valid and contains the "admin" scope, every request to the endpoint returns a 403 Forbidden. You inspect the `SecurityContext` in the debugger and see that the granted authorities assigned to the user are `[SCOPE_read, SCOPE_write, SCOPE_admin]`.

What Spring Security default behavior is causing this mismatch, and how do you resolve it without changing the controller's annotation?

- **A)** Spring Security maps OAuth2 scopes differently depending on the HTTP method. A GET request requires SCOPE_read, while a POST request requires SCOPE_write. The hasRole annotation bypasses this mapping. You must change the endpoint to a GET request.
- **B)** By default, Spring's JwtGrantedAuthoritiesConverter extracts the scope (or scp) claim and prefixes every value with "SCOPE_". However, the hasRole('ADMIN') expression implicitly expects an authority named "ROLE_ADMIN". The mismatch between SCOPE_admin and ROLE_ADMIN causes the 403. You must provide a custom JwtAuthenticationConverter bean to map the scopes to ROLE_ prefixes.
- **C)** The JWT scope claim is a single space-delimited string. Spring Security reads the entire string as a single massive authority: SCOPE_read write admin. You must configure the Auth Server to issue the scopes as a JSON array instead.
- **D)** The @PreAuthorize annotation evaluates roles locally against the database. Because the user does not exist in the Resource Server's physical database, the role check fails. You must implement a UserDetailsService to synchronize the JWT with the local database.

Answer: B

Explanation: This is a classic Spring Security OAuth2 integration hurdle. When Spring parses a JWT, it defaults to looking at the scope or scp claim and turns them into GrantedAuthority objects prefixed with SCOPE_. Conversely, the .hasRole("XYZ") method in Spring Security is hardcoded to prepend ROLE_ to whatever you pass it (checking for ROLE_XYZ). If you want to use .hasRole(), you must write a custom converter bean that extracts the JWT claims and manually creates GrantedAuthority objects with the ROLE_ prefix so they match your method security expressions.

Question 34

You have a Spring Data JPA CustomerRepository. A frontend application sends a PUT request with a DTO to update an existing customer. You map the DTO to a *newly instantiated* Customer entity object, manually set its ID to match the existing customer (e.g., customer.setId(5L)), and then call customerRepository.save(customer).

When you check the SQL logs, you notice that Hibernate executes two queries sequentially for this single operation:

```
SELECT * FROM customer WHERE id = 5;
```

```
UPDATE customer SET ... WHERE id = 5;
```

Why does Spring Data JPA execute that extra SELECT statement, and how do you optimize it to only execute the UPDATE?

- **A)** Hibernate uses the SELECT statement to establish a pessimistic write lock on the row to prevent concurrent modifications during the UPDATE. The fix is to change the transaction isolation level to READ_UNCOMMITTED.
- **B)** When you call save() with an object that is not currently in the Hibernate First-Level Cache (a detached or new instance), and it has an ID, Spring Data's SimpleJpaRepository internally routes the call to entityManager.merge(). The merge operation *must* fetch the current state from the database into memory first so it knows exactly which fields changed before generating the UPDATE. To avoid this, you must query the entity first (findById), update its fields, and let the transaction commit handle the update via dirty checking.
- **C)** The SELECT statement is triggered by the @Version annotation to verify Optimistic Locking hashes. The only way to remove it is to delete the @Version field from the entity,

sacrificing concurrency safety for performance.

- **D)** The extra SELECT is an artifact of the H2 in-memory database used in development. Production databases like PostgreSQL natively support "UPSERT" syntax, which Hibernate automatically utilizes to bypass the initial select.

Answer: B

Explanation: This is the foundational mechanism of JPA's merge() operation. If you create a new object (new Customer()), give it an existing ID, and call save(), Hibernate doesn't just blindly fire an UPDATE statement. It must bring the managed entity into the Persistence Context. It executes a SELECT to get the real entity, copies the properties from your detached object onto the managed entity, and then generates an UPDATE based on the delta. If you want to avoid the SELECT overhead for simple updates, you should either use Dirty Checking (fetch, mutate, let it commit naturally) or execute a custom @Modifying @Query to fire a raw UPDATE statement.

Question 35

You are using Resilience4j to protect a microservice endpoint. You configure both a CircuitBreaker and a TimeLimiter on the same method. The TimeLimiter is set to 2 seconds. The CircuitBreaker is set to open if the failure rate exceeds 50%.

Under load, the external API degrades and starts taking 10 seconds to respond.

You observe that the TimeLimiter is working perfectly—your method throws a TimeoutException after exactly 2 seconds. However, even after hundreds of consecutive timeouts, the CircuitBreaker *never* transitions to the OPEN state, and your application continues to waste threads waiting 2 seconds for every single request instead of failing fast.

What architectural configuration flaw in Resilience4j is causing the Circuit Breaker to ignore the timeouts?

- **A)** The Circuit Breaker only records HTTP 5xx series errors by default. You must explicitly add TimeoutException.class to the recordExceptions list in the Circuit Breaker configuration YAML.
- **B)** This is caused by an incorrect Aspect Ordering. Resilience4j applies patterns based on a strict order. If the Circuit Breaker wraps the TimeLimiter, it will see the TimeoutException and record a failure. If the TimeLimiter wraps the Circuit Breaker, the timeout occurs *outside* the Circuit Breaker's scope, meaning the Circuit Breaker never sees the exception and assumes the call succeeded. You must adjust the resilience4j.circuitbreaker.circuit-breaker-aspect-order property.
- **C)** The TimeLimiter runs the method asynchronously on a separate thread using a CompletableFuture. The Circuit Breaker is bound to the main Tomcat thread via ThreadLocal and cannot monitor exceptions thrown on background threads.
- **D)** The Circuit Breaker evaluates failure rates based on a sliding window of time, not a count. Because the timeouts delay the throughput, the volume of requests per second drops below the minimum-number-of-calls threshold required to trigger state calculation.

Answer: B

Explanation: Resilience4j utilizes Spring AOP under the hood. When you stack multiple annotations (e.g., @CircuitBreaker and @TimeLimiter), Spring creates nested proxies. The order in which they nest is critical. If the TimeLimiter is the inner proxy, it throws the exception, the outer CircuitBreaker catches it, records a failure, and eventually trips. If

the CircuitBreaker is the inner proxy, it executes the method (which takes 10s), but the outer TimeLimiter aborts the outer thread after 2s. The CircuitBreaker never finishes its execution cycle properly and doesn't record the timeout. Resilience4j has a default aspect order (Retry > CB > RateLimiter > TimeLimiter), but misconfigurations or manual decorator usage can break this.

Question 36

You are storing a custom Employee object in a Core Java TreeSet. The Employee class has three fields: id, name, and salary.

To ensure the TreeSet automatically sorts the employees from highest to lowest salary, you make the Employee class implement Comparable<Employee> and override the compareTo method:

Java

@Override

```
public int compareTo(Employee other) {  
    return Double.compare(other.salary, this.salary);  
}
```

You also properly override equals() and hashCode() to evaluate equality strictly based on the unique id field.

You create two distinct employees: John (id=1, salary=50000) and Jane (id=2, salary=50000). You add both to the TreeSet.

When you print the size() of the TreeSet, it outputs 1, and Jane is missing from the collection.

Why did the TreeSet reject a perfectly valid, unique object, and what fundamental contract was violated?

- **A)** The TreeSet implementation requires the compareTo method to return an Integer, but Double.compare returns a double, causing a silent ClassCastException during insertion.
- **B)** A TreeSet strictly ignores the equals() and hashCode() methods. It determines object equality *exclusively* by checking if compareTo(other) == 0. Because John and Jane have the exact same salary, compareTo returns 0, and the TreeSet assumes they are the exact same duplicate object and discards Jane.
- **C)** You inverted the comparison logic to achieve descending order (other.salary, this.salary), which violates the mathematical transitive property of the Comparable interface, corrupting the Red-Black Tree balancing algorithm.
- **D)** The TreeSet successfully stored both objects, but its internal iterator collapses consecutive elements with identical sorting keys to optimize memory. You must call .size(false) to get the true element count.

Answer: B

Explanation: This is a notorious Core Java interview and production trap. HashSet uses hashCode() and equals() to find duplicates. TreeSet (and TreeMap) do *not*. They rely entirely on the compareTo (or Comparator) method. If your compareTo method returns 0, the TreeSet considers the objects absolutely identical and will not add the second one. If

you are sorting a TreeSet by a non-unique field (like salary), your compareTo method must have a secondary fallback (e.g., if salaries are equal, compare their IDs) to ensure it only returns 0 when the objects are truly identical.

Question 37

You want to optimize a heavy Spring MVC endpoint (/api/reports/annual) that generates a massive JSON payload. You register Spring's ShallowEtagHeaderFilter in your configuration.

When a client requests the endpoint, the server returns the JSON and an ETag header (e.g., "03a...8f").

When the client requests the endpoint again 5 minutes later, they send the If-None-Match: "03a...8f" header. The server correctly responds with an HTTP 304 Not Modified and an empty body, saving significant network bandwidth.

However, your APM dashboard shows that the CPU usage and database I/O for the server are exactly identical for both the 200 OK request and the 304 Not Modified request.

Why did the ETag fail to reduce the server-side computational load?

- **A)** ShallowEtagHeaderFilter is implemented at the Tomcat Servlet layer. It physically allows the entire Spring MVC Controller to execute, queries the database, and serializes the full JSON string into memory. It then hashes that JSON to compute the ETag. If the hash matches the client's header, it drops the payload and sends the 304. It saves network bandwidth, not server compute time.
- **B)** The client's HTTP library is incorrectly formatting the If-None-Match header by enclosing the hash in double quotes. Spring MVC cannot parse the quotes, assumes it is a cache miss, regenerates the data, and then strips the quotes at the very last millisecond.
- **C)** The database transaction manager overrides the ETag filter. Because the method is annotated with @Transactional, Spring forces the Hibernate Session to execute the SQL queries to guarantee that the data hasn't changed before generating the 304.
- **D)** ETags are strictly designed for static assets (HTML/CSS/JS) served from the src/main/resources/static directory. When applied to dynamic @RestController endpoints, the filter defaults to a "cache-bust" mode to prevent stale data anomalies.

Answer: A

Explanation: The word "Shallow" is the key here. Spring's ShallowEtagHeaderFilter does exactly what it says on the tin. It intercepts the response *after* your controller has done all the heavy lifting (DB queries, complex calculations, JSON serialization). It calculates an MD5 hash of that finalized response body. If the hash matches the request header, it stops the body from being sent over the network. It is fantastic for saving bandwidth on mobile devices, but it does absolutely nothing to save CPU or Database load. To save server compute, you must implement "Deep ETags" (calculating the hash manually in the controller based on a DB updated_at timestamp before doing the heavy lifting) or use @Cacheable.

Question 38

You are deploying your Spring Boot application to Kubernetes. You configure the Kubernetes Liveness Probe to hit /actuator/health/liveness and the Readiness Probe to hit /actuator/health/readiness.

Your application relies on an external RabbitMQ message broker. During a network blip, the connection to RabbitMQ drops.

Spring Boot Actuator detects the RabbitMQ failure and changes the overall application health status to DOWN.

Because the application is DOWN, Kubernetes kills the pod and restarts it, causing active HTTP user sessions to be violently terminated.

How should you correctly configure Spring Boot Actuator so that a RabbitMQ failure marks the application as "unready" (stops routing new HTTP traffic to it) but does NOT mark it as "dead" (preventing Kubernetes from killing the pod)?

- **A)** You must write a custom ApplicationRunner bean that intercepts the RabbitAdmin failure events and manually triggers an HTTP 503 response on the Gateway router, bypassing Actuator entirely.
- **B)** In your application.yml, you must move the rabbit health indicator from the global health group into the readiness group exclusively, and remove it from the liveness group. This ensures the liveness probe (which dictates if the pod should be killed) ignores RabbitMQ, while the readiness probe (which dictates routing) monitors it.
- **C)** Change the Kubernetes Liveness Probe failureThreshold from the default of 3 to 100. This gives the application enough time to establish a new TCP handshake with RabbitMQ before Kubernetes issues the SIGKILL command.
- **D)** Disable the RabbitMQ health check entirely (management.health.rabbit.enabled=false). Actuator cannot safely differentiate between liveness and readiness states for stateful TCP connections.

Answer: B

Explanation: This is a crucial DevOps/Spring Boot integration concept. A failed Liveness Probe tells Kubernetes "I am in a corrupted state, shoot me in the head and restart me." A failed Readiness Probe tells Kubernetes "I am busy or temporarily disconnected, stop sending me traffic, but leave me alive." If a downstream service (like DB or RabbitMQ) goes down, you usually don't want your app to be killed; you just want it to stop taking traffic until the dependency recovers. By default, Spring Boot 2.3+ natively supports Kubernetes Probes. You can organize health indicators into "Groups". You configure your YAML to include the Rabbit/DB indicator in the readiness group and exclude it from the liveness group.

Question 39

You are writing a complex reporting feature using Spring Data JPA. The query is too complex for JPQL, so you write a native SQL query:

Java

```
@Query(value = "SELECT c.id, c.name, SUM(o.total) as revenue FROM customers c JOIN orders o ON c.id = o.customer_id GROUP BY c.id", nativeQuery = true)
```

```
Page<CustomerRevenueDTO> getTopCustomers(Pageable pageable);
```

When you execute this method, Spring Boot throws an InvalidDataAccessApiUsageException immediately at runtime.

Why does Spring Data JPA refuse to execute this paginated native query, and how do you resolve it?

- **A)** Native queries return raw `Object[]` arrays, which cannot be automatically mapped directly into a Java DTO (`CustomerRevenueDTO`) without implementing a custom `AttributeConverter`.
- **B)** Spring Data JPA cannot reliably parse complex native SQL (especially with `GROUP BY` or subqueries) to automatically generate the `COUNT(*)` query required to calculate the total number of pages for the `Page` object. You must explicitly provide a `countQuery` attribute within the `@Query` annotation.
- **C)** The `Pageable` interface injects ANSI SQL `OFFSET` and `FETCH NEXT` keywords. If you are using a MySQL database older than version 8.0, it rejects the syntax. You must switch to `Slice<CustomerRevenueDTO>`.
- **D)** You cannot use aliases (like `c` and `o`) in a native query if you are passing a `Pageable` object, because Spring's internal SQL interceptor attempts to prepend the table names to the alias variables, resulting in malformed SQL.

Answer: B

Explanation: When you ask Spring Data to return a `Page<T>`, it must execute two queries: your actual query (with `LIMIT/OFFSET` applied) and a `COUNT` query to figure out `totalPages` and `totalElements`. For standard derived queries or simple JPQL, Spring generates this count query easily by stripping out the `SELECT` and inserting `SELECT count(1)`. But for Native SQL—especially queries involving `GROUP BY`, `DISTINCT`, or nested joins—it is mathematically impossible for Spring to reliably guess how to rewrite it into a valid count query. Therefore, it aborts. You must provide it manually: `@Query(value = "...", countQuery = "SELECT count(*) FROM (your complex query)", nativeQuery = true)`.

Question 40

You have a complex business process managed by a Spring `@Service`.

Method A is annotated with `@Transactional`. It performs some database writes and then calls Method B, which is located in a different Spring Service and is also annotated with `@Transactional` (using default propagation).

Method B performs a database write but encounters a constraint violation and throws a `DataIntegrityViolationException` (a `RuntimeException`).

To prevent the entire process from failing, you wrap the call to Method B inside a try-catch block within Method A:

Java

```
// Inside Method A
```

```
try {  
    serviceB.methodB();  
} catch (DataIntegrityViolationException e) {  
    log.warn("Method B failed, but continuing Method A");  
}
```



```
// ... finish Method A work
```

Method A catches the exception, completes its remaining logic without further errors, and exits. However, instead of committing Method A's work, Spring throws a massive `UnexpectedRollbackException`: Transaction silently rolled back because it has been marked as rollback-only.

Why did Spring forcefully roll back the entire transaction even though you successfully caught and handled the exception in Java?

- **A)** The JDBC connection pool automatically severs the physical network connection to the database the millisecond a constraint violation occurs, rendering the remainder of Method A's JDBC commands invalid.
- **B)** The default propagation for Method B is `REQUIRED`, meaning it joins Method A's existing transaction. When Method B throws the `RuntimeException`, its AOP proxy catches it, flags the shared transaction resource as rollback-only, and rethrows it. Even though Method A catches the Java exception, the global database transaction is already permanently poisoned. When Method A finishes and attempts to commit, the Transaction Manager sees the poisoned flag and throws the error.
- **C)** Catching an exception inside a `@Transactional` block violates the Java Transaction API (JTA) specification. The Spring container detects the catch block using bytecode analysis and intentionally throws an `UnexpectedRollbackException` to enforce best practices.
- **D)** The `log.warn()` statement triggers a synchronous write to the filesystem. This blocks the main thread long enough for the database's internal transaction timeout monitor to expire, killing the transaction behind the scenes.

Answer: B

Explanation: This is arguably the most common and confusing `@Transactional` trap in Spring. When Method B joins Method A's transaction, they share the exact same physical database transaction. When Method B throws a `RuntimeException`, it passes through Method B's transactional proxy. The proxy says, "Ah, an exception! I must mark this transaction for rollback." It sets a flag on the shared transaction. Method A catches the exception and says, "Everything is fine, let's commit!" But when Method A's proxy tries to commit, the overarching Transaction Manager sees the rollback-only flag and panics. It throws `UnexpectedRollbackException` because Method A asked for a commit, but a commit is no longer possible. To fix this, Method B must be marked with `REQUIRES_NEW` so its rollback does not poison Method A's transaction.